Apellidos: ..... Nombre: ..... Grup: .....

Examen final IC (*Parte 2)*

- Duración del examen: 2:30 horas.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución del examen se publicará en Atenea mañana por la mañana.

Nota: La indicación de la puntuación de los ejercicios es sobre 10 puntos, pero esta parte del examen final solo representa 5 puntos de la nota del examen final.

Ejercicio 1 *(1.9 puntos)*

El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina para ser ejecutado en el SISC Von Neumann, situando la sección de datos (`.data`) a partir de la dirección **`0x3200`** y la sección de código (`.text`) a partir de la dirección **`0xC500`**.

```
.data
        long=4
        A: .word 0x00EF, 0xABA0, 0x0100, 0xFFBE, 0x1234
        B: .word 0xBEEF, 0xDEAD, 0xCAFE, 0xBABE, 0xACDC
        res: .space long
.text
        MOVI  R1, lo(A)
        MOVHI R1, hi(A)
        MOVI  R2, lo(B)
        MOVHI R2, hi(B)
        MOVI  R3, long
        MOVI  R7, 15
        SHL   R7, R7, R3
        MOVI  R0, 0
loop:   LD    R4, 0(R1)
        LD    R5, 0(R2)
        XOR   R6, R4, R5
        AND   R6, R6, R7
        BNZ   R6, et2
et1:    ADDI R0, R0, 1
        BZ    R6, cont
et2:    ADDI R0, R0, -1
cont:   ADDI R1, R1, 2
        ADDI R2, R2, 2
        ADDI R3, R3, -1
        BNZ R3, loop
        ST long(R2), R0
.end
```

a) Una vez ensamblado y cargado el programa en la memoria, ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.4 puntos)*

res= 0x ____ **cont=** 0x ____

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@ memoria y contenido
`SHL R7, R7, R3`	Mem_w[0x ____] = 0x
`BNZ R3, loop`	Mem_w[0x ____] = 0x
`LD R5, 0(R2)`	Mem_w[0x ____] = 0x

c) Una vez ejecutado el programa en el SISC Von Neumann ¿Cuál es la dirección de la memoria donde ha escrito la última instrucción (`ST long(R2),R0`) y cuál es su contenido? *(0.6 puntos)*

Mem_w[0x ____] = 0x ____

d) Si se ejecutase el programa en el computador en la SISC Harvard multiciclo, indicad el número total de instrucciones que ejecutaría, así como cuántas son lentas y cuántas son rápidas. *(0.3 puntos)*

N_{total} = ____ **N_{lentas} =** ____ **$N_{rápidas}$=** ____

Ejercicio 2 *(2 puntos)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UCG en el anexo. Suponed que el contenido de todos los registros, `Rk` para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido del registro IR. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.4 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Valor del IR** (en hexadecimal)
movhi	`MOVHI R4,0xBC`	0x
addr	`LD R4, -2(R3)`	0x
jalr	`JALR R5, R0`	0x
addi	`ADDI R3, R2, 5`	0x

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.8 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
movhi	`MOVHI R4,0xBC`	0x
addr	`LD R4, -2(R3)`	0x
jalr	`JALR R5, R0`	0x
addi	`ADDI R3, R2, 5`	0x

c) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT). *(0.8 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
		@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	LdIr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
movhi	`MOVHI R4,0xBC`																			
addr	`LD R4, -2(R3)`																			
jalr	`JALR R5, R0`																			
addi	`ADDI R3, R2, 5`																			

Ejercicio 3 *(1.2 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x2C78`, el contenido de todos los registros pares es `0x2222` y el de los registros impares es `0x3333`, el byte contenido en todas las direcciones pares de la memoria es `0x89` y el de todas las impares es `0xA5` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x…]=0x... y/o MEM_w[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Apellidos: …… Nombre: …… Grup: ……

Instrucción a ejecutar	Cambios en el estado del computador
`STB 7(R1), R5`	
`JALR R0, R1`	
`MOVI R2,0x95`	
`IN R3, 14`	

Ejercicio 4 *(1 punto)*

Se ha conectado a la UPG un dispositivo externo de entrada que nos envía valores y que tiene el registro de status en la dirección 4 del espacio de direccionamiento de entrada y el de datos en la 5. También se ha conectado un dispositivo externo de salida que tiene el registro de status en la dirección 6 del espacio de direccionamiento de entrada y el de datos en la dirección 7 del espacio de direccionamiento de salida. Ambos dispositivos tienen un efecto lateral en la lectura/escritura del dato sobre su registro de estado.

Se desea implementar un algoritmo en lenguaje SISA que lea un dato (que se interpreta como un número natural) del dispositivo de entrada y lo envíe al dispositivo de salida. Si el dato recibido es impar lo convierte al par inmediatamente anterior. Si el dato recibido es par se deja igual. Este proceso debe realizarse indefinidamente hasta que el dato enviado al dispositivo de salida sea el 0. Por ejemplo. Si el dato recibido es el valor 26 (par) se envía el mismo valor al dispositivo de salida, pero si el dato recibido es el 29 (impar) se envía el valor 28. Escribid el programa en código ensamblador SISA para que realice esta función. Se valorará la claridad y brevedad del código. El código no debe ocupar más de 20 instrucciones.

Ejercicio 5 *(1.6 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 80 u.t.

T_p(ROM_OUT) = 150 u.t.

T_p(MUX-2-1) = 40 u.t.

T_p(MUX-4-1) = 80 u.t.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 350 u.t. // Tiempo de lectura del banco de registros

T_p(ALU-slow) = 750 u.t. // Tp de la ALU para las operaciones/funciones lentas: ADD, SUB, CMP*.

T_p(ALU-quick) = 400 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de ADD, SUB, CMP*.

T_{acc}(64KB MEMORY) = 800 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

a) T_c correspondiente al nodo de **F** (Fetch).

T_c=

b) T_c correspondiente al nodo de **Addi**.

T_c=

c) T_c correspondiente al nodo de **Stb**.

T_c=

d) T_c correspondiente al nodo de **Addr**.

T_c=

Ejercicio 6 *(2.3 puntos)*

Debido a que es muy frecuente en programas tener que enviar un vector de valores almacenado en la memoria a un dispositivo de salida, se ha decidido crear una nueva instrucción (**LDOUT+**) que haga la mayor parte de este trabajo. Esta nueva instrucción leerá el valor almacenado en la posición de memoria indicada por un registro y lo enviará a un dispositivo de salida. Además, actualizará el valor del registro para la lectura del siguiente valor.

La nueva instrucción tiene el formato y codificación, la sintaxis ensamblador y la semántica siguientes:

Codificación:	`1111 aaa x nnnnnnnn`
Sintaxis:	`LDOUT+ N8, Ra`
Semántica:	`PC=PC+2; OUTPUT[N8]=MEM`$_w$`[Ra]; Ra=Ra+2`

Donde `Ra` es la dirección de memoria donde está almacenado el primer elemento del vector y `N8` el puerto de datos del dispositivo. Por ejemplo, supongamos que deseamos enviar un vector A de 100 valores de tipo word a un dispositivo, que tiene el registro de status en el puerto 5 y el de datos en el 6. El código SISA original y con la nueva instrucción para realizar la tarea serían los siguientes:

Código original

```
          MOVI  R0, 100
          MOVI  R1, lo(A)
          MOVHI R1, hi(A)
bucle:    IN    R7, 5
          BZ    R7, bucle
          LD    R7, 0(R1)
          OUT   6, R7
          ADDI  R1, R1, 2
          ADDI  R0, R0, -1
          BNZ   R0, bucle
```

Código con la nueva instrucción

```
          MOVI   R0, 100
          MOVI   R1, lo(A)
          MOVHI  R1, hi(A)
bucle:    IN     R7, 5
          BZ     R7, bucle
          LDOUT+ 6, R1
          ADDI   R0, R0, -1
          BNZ    R0, bucle
```

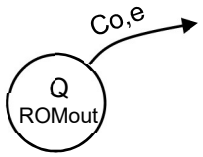
Apellidos: ..... Nombre: ..... Grup: .....

Para añadir esta nueva instrucción no es necesario modificar el hardware del procesador SISC Von Neumann. Completad el diseño del computador para que ejecute, además de las 25 instrucciones originales SISA, la nueva instrucción **LDOUT+** sin modificar el hardware y sólo modificando el contenido de las ROM's. Se pide:

Se pide:

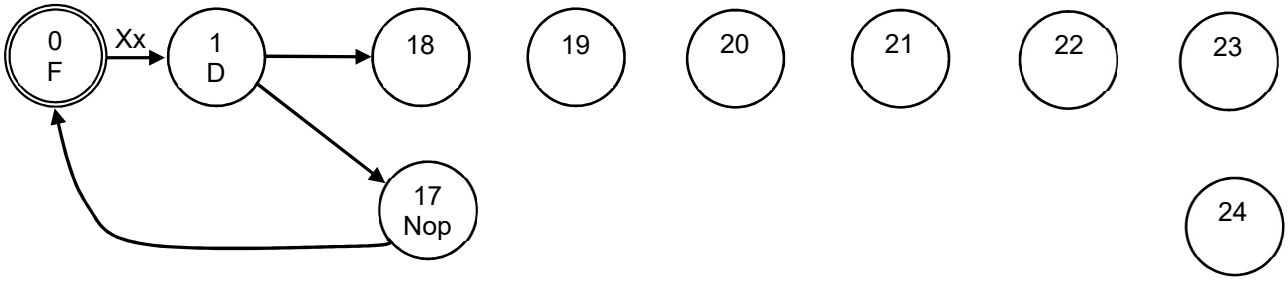
a) Completad el fragmento del grafo de estados de la figura correspondiente al circuito secuencial de la unidad de control. Se da la leyenda del grafo y los nodos necesarios para ejecutar la nueva instrucción, pero faltan arcos y etiquetas. Dibujad todos los arcos que faltan y todas las etiquetas. En número de nodos necesarios para ejecutar la nueva instrucción dependerá de la solución que propongáis, pero no serán más de 9 (incluyendo F y D) y deberá ser coherente con las acciones que pongáis en el apartado b). Los nodos que os sobren podéis dejarlos sin conectar. *(0.1 puntos)*

Leyenda:



CO: Código de operación de la Instrucción, I_{15} I_{14} I_{13} I_{12} (en hexadecimal)
e: Bit de extensión del código de operación (I_8)
Q: Estado (en decimal)
ROMout: Mnemotécnico de salida

Grafo:



b) Completad el contenido de la siguiente tabla que indica, mediante una fila para cada nodo del grafo de estados de la unidad de control, la acción (o acciones en paralelo) que se realiza en el computador en cada uno de los ciclos/nodos que requiere la ejecución de la nueva instrucción (Fetch, Decode, y los ciclos/nodos de la ejecución propiamente dicha). Para especificar las acciones usad el mismo lenguaje de transferencia de registros que en la documentación. En número de filas/nodos de la tabla a rellenar dependerá de la solución que propongáis y deberá ser coherente con el apartado a) y c). Nota: Existe una solución solo con 6 nodos (F, D y 4 nodos nuevos). *(0.8 puntos)*

Nodo	Mnemotécnico	Acciones
E0	F	IR ← Mem_w[PC] // PC ← PC+2
E1	D	RX ← Ra // RY ← Rb // R@ ← PC+SE(N8)*2
E18		
E19		
E20		
E21		
E22		
E23		
E24		

Si la solución propuesta en el apartado b) no permite ejecutar correctamente la nueva instrucción, no se corregirán los apartados a) ni c).

c) Completad (poniendo 0, 1 o x en cada bit) la siguiente tabla que especifica, mediante una fila por cada nuevo nodo añadido al grafo de estados de la unidad de control, la dirección (en decimal) de la ROM_OUT y su contenido para que se ejecute correctamente la nueva instrucción. **Poned x siempre que el valor de un bit no importe**. En la columna de la derecha poned el mnemotécnico que habéis usado para nombrar cada nuevo nodo en el grafo del apartado b). *(0.8 puntos)*

@ROM	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	$P/I/L/A_1$ $P/I/L/A_0$	OP_1 OP_0	MxN_1 MxN_0	MxF	F_2 F_1 F_0	$Mx@D_1$ $Mx@D_0$	**Nodo (Mnemo)**
18																			
19																			
20																			
21																			
22																			
23																			
24																			

d) Completad la dirección o contenido, según corresponda, de la ROM_Q+ del SISC Von Neumann. *(0.2 puntos)*

`ROM_Q+[0x02D]=` `0x` ____ `ROM_Q+[` `0x` ____ `]=0x0F`

e) Suponiendo que la ejecución de la nueva instrucción se realiza en 6 ciclos (F, D y 4 nodos nuevos), y asumiendo que el dispositivos está siempre disponible (registro de status a 1) para recibir un dato. Indica cuantas instrucciones se ejecutarían y cuantos ciclos se tardaría en ejecutar el programa de ejemplo del inicio del enunciado si ejecutásemos el código original y si ejecutásemos el programa con la nueva instrucción. *(0.4 puntos)*

Número instrucciones código original: ____ Número instrucciones con la nueva instrucción: ____

Ciclos de ejecución código original: ____ Ciclos de ejecución con la nueva instrucción: ____